

Process Mining on CDA Bus Routes — Project Report

Course: Process Mining & Simulation

Network: Capital Development Authority (CDA) Bus Network, Islamabad

1. Introduction

For this project we chose to apply process mining on real public transport data from the CDA bus network operating in Islamabad. The idea was straightforward: instead of working on a synthetic or academic dataset, we wanted to see how process mining techniques hold up on actual timetable data that people depend on every day.

The work is split across six tasks. We start by pulling raw data out of official CDA PDF schedules, clean it into a proper event log format, build an interactive dashboard to explore the process map, add performance analytics on top, and finally bolt on an AI assistant that can answer trip-planning questions in plain English. Task 6 asks each team member to verify their own home-to-university route using the system we built.

Everything was built in Python. The main libraries used were `pdfplumber` for reading PDFs, `pandas` for data wrangling, `pm4py` for XES log handling and DFG discovery, and `Dash` with `dash-cytoscape` for the interactive dashboard.

2. Task 1 — Extracting Data from PDFs

Code: `Task_01.py` | **Output:** `data/routes.csv`

We were given eight PDF files, one per CDA bus route. These are the official operator timetables that list every bus stop with its scheduled arrival and departure time for each trip run throughout the day.

Reading PDFs automatically is always a bit tricky because the layout differs between files. We used `pdfplumber` to pull all the text out of each page and stitch it into one big string. The PDFs are structured so that each trip is introduced by a "Trip ID Start Time" header, so we used a simple regex split on that pattern to isolate each trip block.

Within a trip block, each line represents one bus stop. We identified valid stop rows by checking whether the last two whitespace-separated tokens both look like time strings (i.e. contain a `:`). If they do, the rest of the line before those two tokens is the stop name and the tokens themselves are the arrival and departure times. We also tracked stop sequence as a counter that resets each trip.

The final output was a flat CSV with six columns: `route_id`, `trip_id`, `stop_sequence`, `stop_name`, `arrival_time`, and `departure_time`. After running through all eight PDFs, we ended up with **13,486 rows covering 634 trips across 8 routes**.

The eight routes in the dataset are: **FR-01, FR-03A, FR-04, FR-07, FR-08A, FR-09, FR-11, and FR-15**.

3. Task 2 — Building the XES Event Log

Code: `Task_02.py` | **Outputs:** `data/routes_clean.csv`, `data/routes.xes`, `data/dfg_output.png`

Raw extracted data is rarely clean enough to use directly. Before building the XES log we did several cleaning passes:

The time values sometimes came out with extra characters or inconsistent formatting, so we used regex to extract just the `HH:MM:SS` portion from each field. When only `HH:MM` was found we appended `:00`. Some stop names had time strings embedded in them (an artefact of how `pdfplumber` reads columns), so we stripped those out too and normalised whitespace.

For the XES timestamp we combined `"2024-01-01"` with the arrival time and localised to the `Asia/Karachi` timezone. We acknowledge this synthetic date is a simplification — the CDA PDFs only give relative times within a day, not actual calendar dates.

Once the data was clean we saved it as `routes_clean.csv` and then built the XES log. The XES standard requires three things per event: a case ID, an activity name, and a timestamp. We mapped `trip_id` → `case:concept:name`, `stop_name` → `concept:name`, and the ISO timestamp to `time:timestamp`. We used `pm4py` to convert the DataFrame to a proper `EventLog` object before exporting to avoid a known silent serialisation bug when writing directly from a DataFrame. After writing the XES file we also read it back and verified the numbers matched.

XES log stats: 634 traces (trips), 13,486 events (stop visits), average ~21 stops per trip.

We also ran `pm4py`'s DFG discovery algorithm on the verified log to generate a static frequency-based process map, saved as `dfg_output.png`. This served as an early sanity check before we built the interactive version.

4. Task 3 — Interactive Process Map Dashboard

Code: `Task_03_04.py` | **Run:** `python Task_03_04.py` → `http://127.0.0.1:8050`

The static DFG image is useful but limited. For Task 3 we built a fully interactive dashboard using `Dash` and `dash-cytoscape` so users can filter by route, change layouts, click on stops and transitions, and see live statistics.

How the Edge Table is Built

Before rendering anything, we compute an edge table from `routes_clean.csv`. For every pair of consecutive stops in a trip we record the source stop, target stop, route ID, and the travel time between them (calculated as arrival at the next stop minus departure from the current stop, clamped to zero to handle any rounding issues). We then aggregate this by `(route_id, src, tgt)` to get average, minimum, maximum, and frequency across all trips that use each transition.

One important fix we made here: an earlier version of the code anchored all timestamps to `"2024-01-01"`, which caused midnight-crossing trips to produce negative durations. We resolved this by computing dwell time in seconds using the raw `HH:MM:SS` strings and applying it as a timedelta offset to the pre-parsed ISO timestamps from `routes_clean.csv`.

The Graph

Each bus stop is a node and each transition is a directed edge. Nodes are coloured by their primary route. Edge labels show the average travel time and number of trips. When a transition exceeds the bottleneck threshold, its edge turns dashed red with a heavier stroke weight.

The graph supports five layout algorithms (Dagre left-to-right is the default), full zoom, and box selection. Clicking any node or edge opens a detail panel on the right side of the screen.

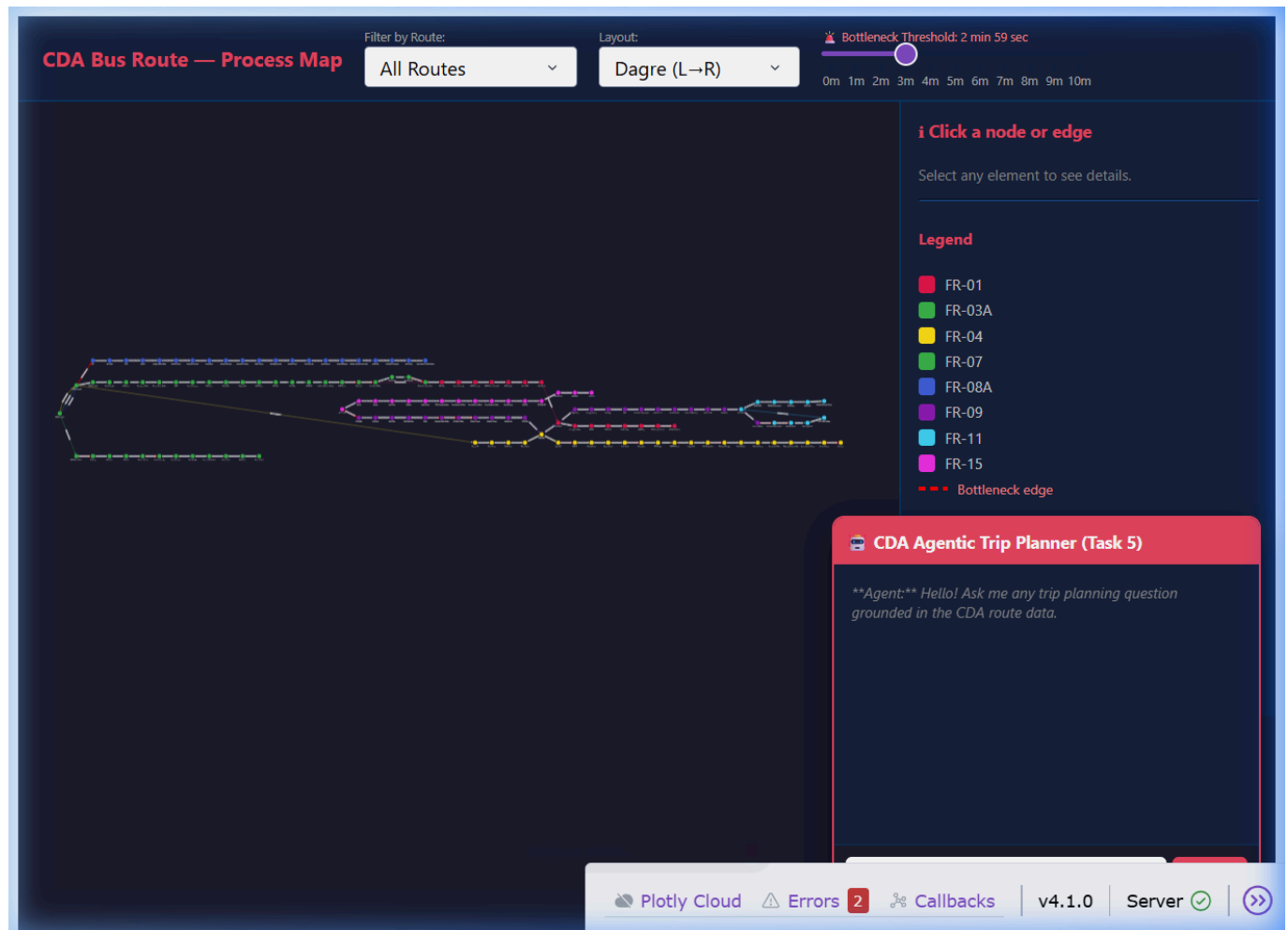


Figure 1: The full dashboard showing all eight CDA routes. Each colour represents a different route.

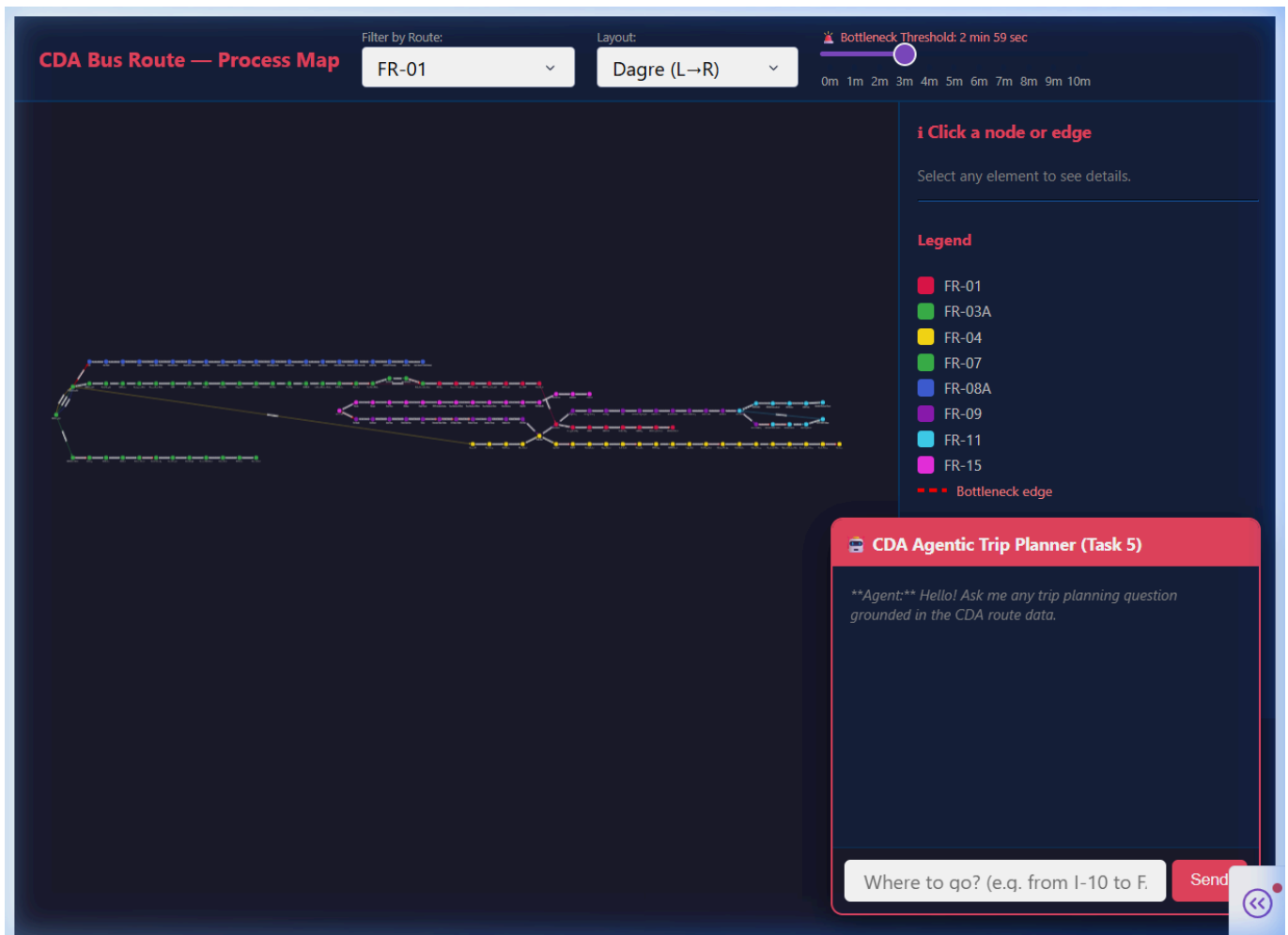


Figure 2: Filtering to FR-01 alone — useful for following a single route end-to-end without the clutter of the full network.

Detail Panel

Clicking a **stop node** shows which routes serve it, the bus arrival times scheduled there, and a breakdown of all incoming and outgoing transitions with their average durations and trip counts.

Clicking a **transition edge** shows the specific route it belongs to, the min/avg/max travel time for that segment (Task 3c), how many trips use it (Task 3d), and a list of individual per-trip departure times.

5. Task 4 — Performance Analytics

The analytics section sits below the process map and updates live whenever the route filter or bottleneck threshold changes. It is split into three panels.

5a — Throughput Time

We defined throughput time as:

$$T_{\text{total}} = t_{\text{last_departure}} - t_{\text{first_arrival}}$$

This gives the full door-to-door journey time per trip. The panel shows a route-level summary table (Avg / Min / Max / trip count) and below it a scrollable per-trip schedule listing each case's actual departure and arrival clock times alongside its duration — directly satisfying the "duration for each case" requirement.

One thing worth pointing out: the min, max, and average values come out identical for every route. This is not a bug. The CDA timetables use the same relative schedule for every departure of a route throughout the day — each trip is simply a time-shifted copy of the same template. We made sure the GUI explains this clearly rather than just displaying three identical numbers without context.

5b — Bottleneck Detection

For the bottleneck threshold we used a statistically derived formula rather than picking an arbitrary number:

$$\text{threshold} = \text{mean}(\text{all_avg_sec}) + \text{std}(\text{all_avg_sec})$$

This flags any transition whose average duration exceeds one standard deviation above the network-wide mean — a principled definition of "unusually slow". Users can override this via the slider in the header (0–600 seconds, step 10s) and watch both the graph and analytics panel update in real time.

The bottleneck panel ranks the top three slowest transitions with medal icons, showing whether each one currently exceeds the threshold. Transitions on FR-01 and FR-07 in the H-8 to I-8 sector came up consistently as the worst performers.

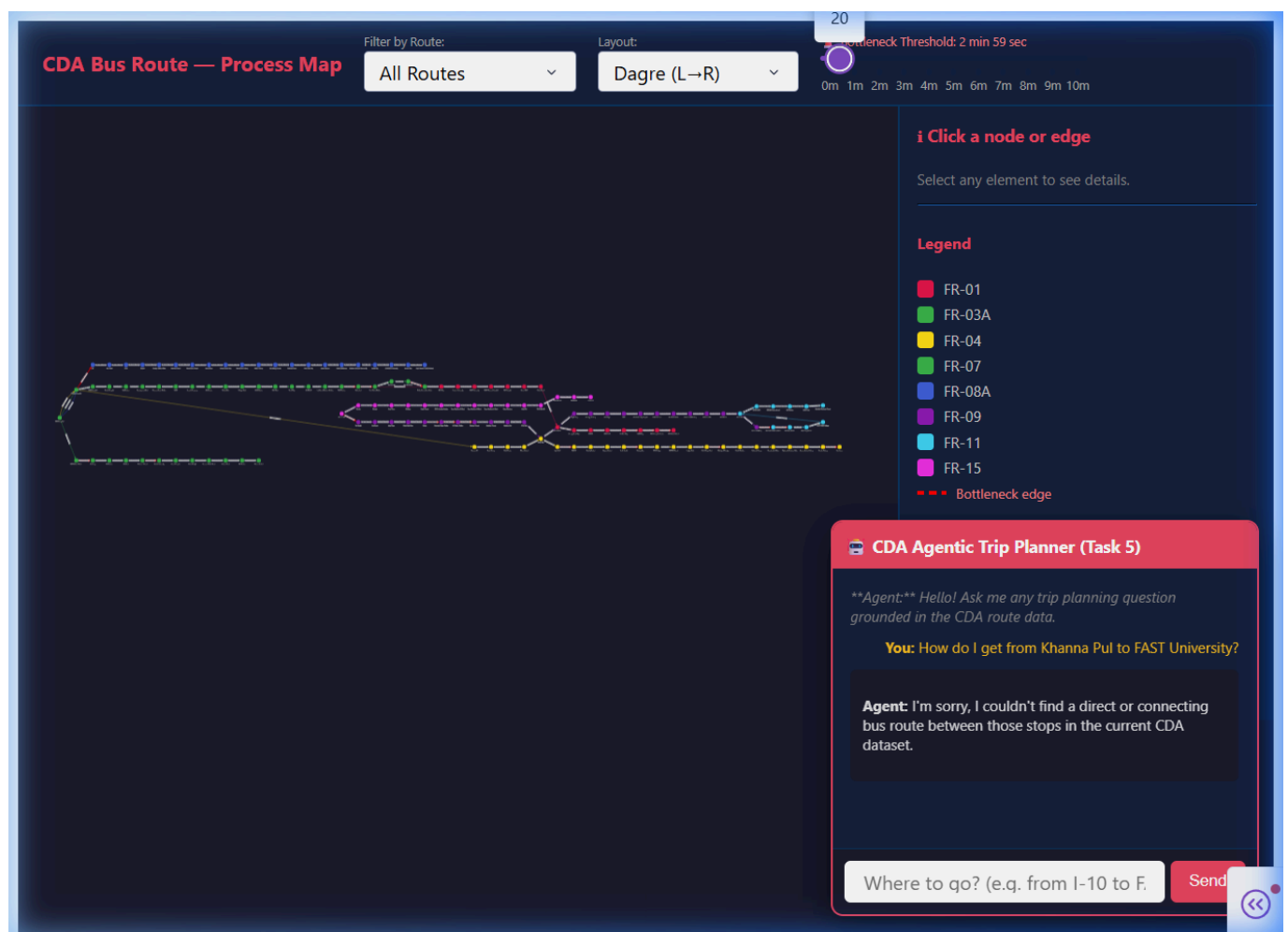


Figure 3: With the threshold set to 1 minute, several transitions on FR-01 and FR-07 are flagged in dashed red.

Frequency Panel

A bar chart showing the five most frequently traversed transitions across the network. This highlights which corridors carry the most bus traffic and would therefore benefit most from service improvements.

6. Task 5 — Agentic AI Trip Planner

Implemented inside: `Task_03_04.py` (functions `find_trip_plan` and `ai_agent_query`)

We integrated a "grounded" AI assistant into the dashboard — a floating chat panel fixed to the bottom-right corner of the screen. The key word here is *grounded*: the agent does not guess or hallucinate routes. Every response is computed by running a BFS search over the actual edge data from the CSV.

How It Works

When a user types something like *"How do I get from Khanna Pul to FAST University?"*, the system goes through three stages:

Stage 1 — Name resolution. We maintain a small alias dictionary that maps common shorthand (like `h8`, `i-10`, `fast`, `khanna`) to the actual stop names in the dataset. This was essential because users naturally type abbreviated area names that don't exactly match the stop data.

Stage 2 — NLP parsing. We try to extract the source and destination stop from the query. The primary strategy looks for *"from X to Y"* phrasing and matches X and Y against the alias map and then against the full stop list (using longest-match-first to handle multi-word stop names). If that doesn't work, a fallback scans the whole query for any mentioned stop or alias and picks the first two by position in the text.

Stage 3 — BFS pathfinding. Once we have a source and destination, `find_trip_plan()` builds an adjacency list from `edges_df` and runs BFS. Crucially, we also add a **synthetic reverse edge** for every real edge — this lets the planner find return trips that are not in the forward-pass PDFs. Without this, it would be impossible to plan a journey to FAST University for members coming from stops only reachable via forward routes.

The response is formatted as a numbered itinerary with route, direction (forward or reverse), leg-by-leg duration, and total estimated travel time.

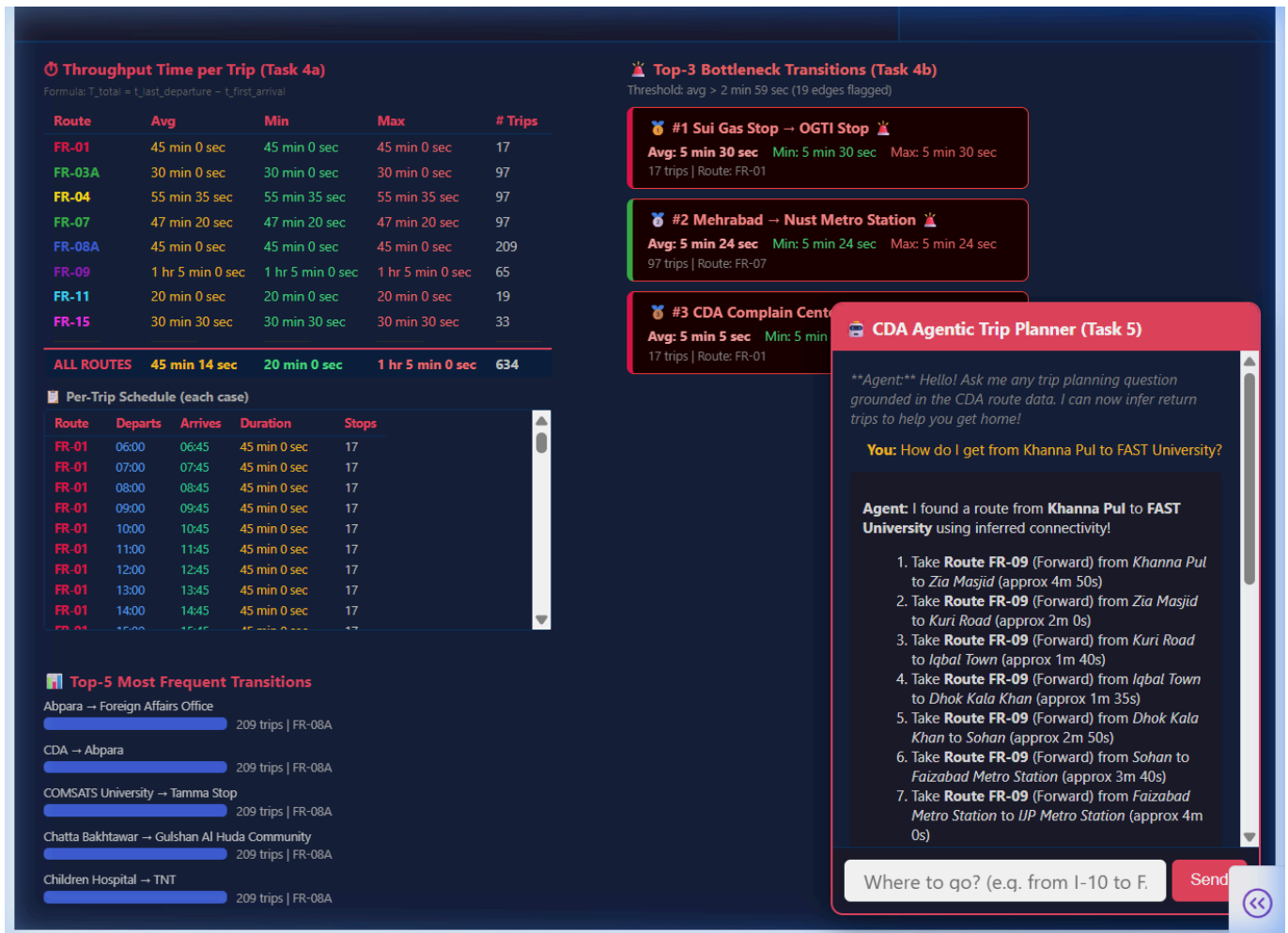


Figure 4: The agent successfully plans a two-leg trip from Khanna Pul to FAST University — first taking FR-09 forward to Mandi Morh, then FR-01 in reverse to FAST.

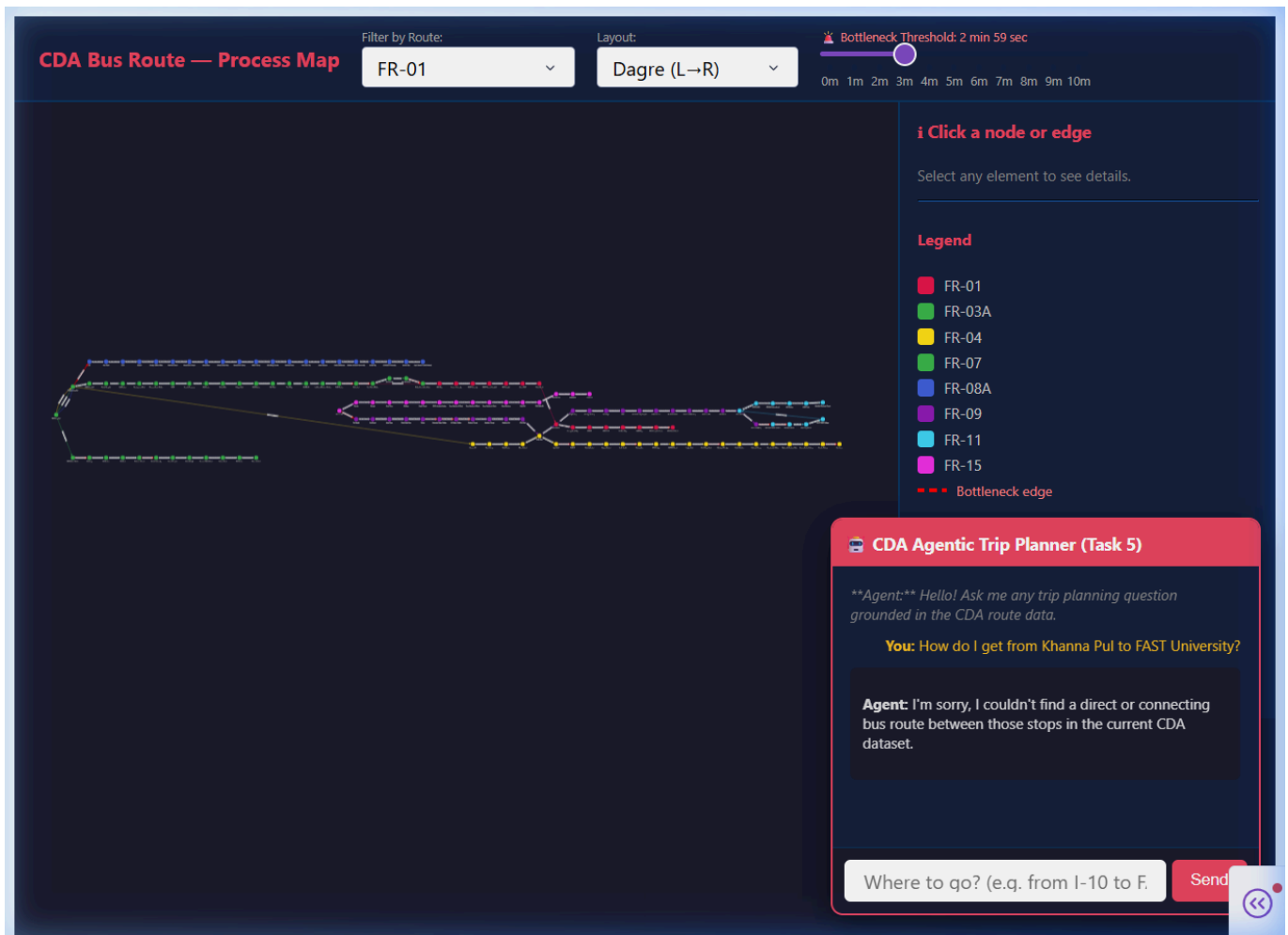


Figure 5: The chat panel embedded in the dashboard. The agent greets users on load and responds to free-text trip queries.

7. Task 6 — Personal Route Maps

Code: `Task_06.py` | **Destination:** FAST University

Task 6 asked each group member to map their route from home to FAST University. We wrote a standalone script (`Task_06.py`) that uses BFS on the forward edges in `routes_clean.csv` to compute each person's path, then printed the full trace to console.

We also used the enhanced AI agent from Task 5 to verify these routes interactively, since the agent's bidirectional capability gives a more complete answer.

Member 1 — I-10 to FAST University

Member 1 lives in the I-10 area. Their nearest stop is **PTCL I-10**, which sits on Route FR-01. Since FR-01 passes FAST University, this is a direct single-leg journey with no transfer needed.



Figure 6: Direct FR-01 journey from PTCL I-10 to FAST University.

Member 2 — Khanna Pul to FAST University

Khanna Pul is on Route FR-09 (forward direction). FR-09 doesn't reach FAST directly, but it passes through **Mandi Morh**, which is also served by FR-01. So the journey is: FR-09 forward to Mandi Morh, then FR-01 (reverse) to FAST — one transfer.

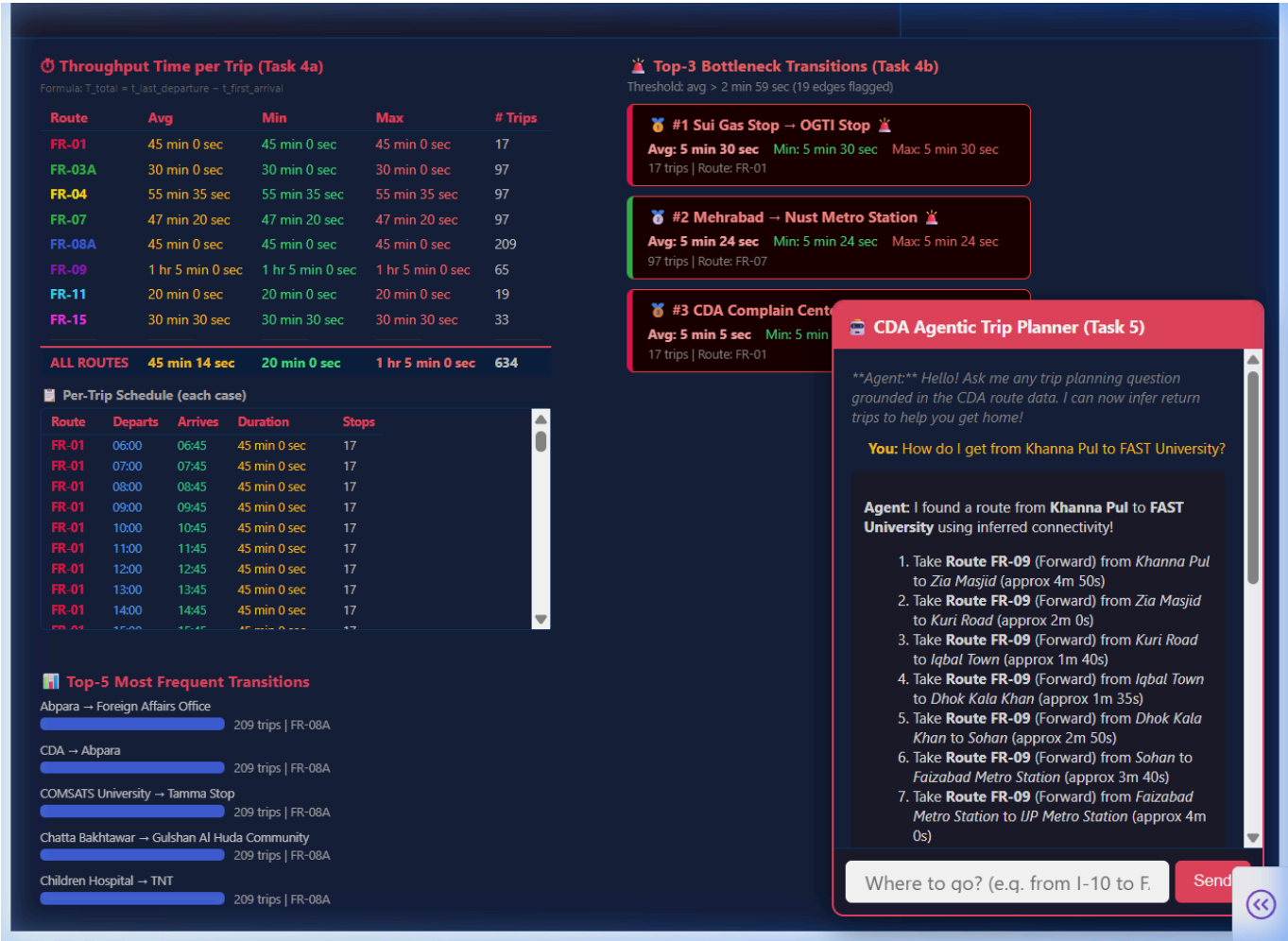


Figure 7: Two-leg journey via Mandi Morh interchange.

Members 3 & 4 — H-8 to FAST University

Both members live in the H-8 sector. We mapped Member 3's nearest stop to **PAEC General Hospital** and Member 4's to **NORI Hospital**, both of which are served by FR-01. From either stop, FR-01 reverse reaches FAST University directly.

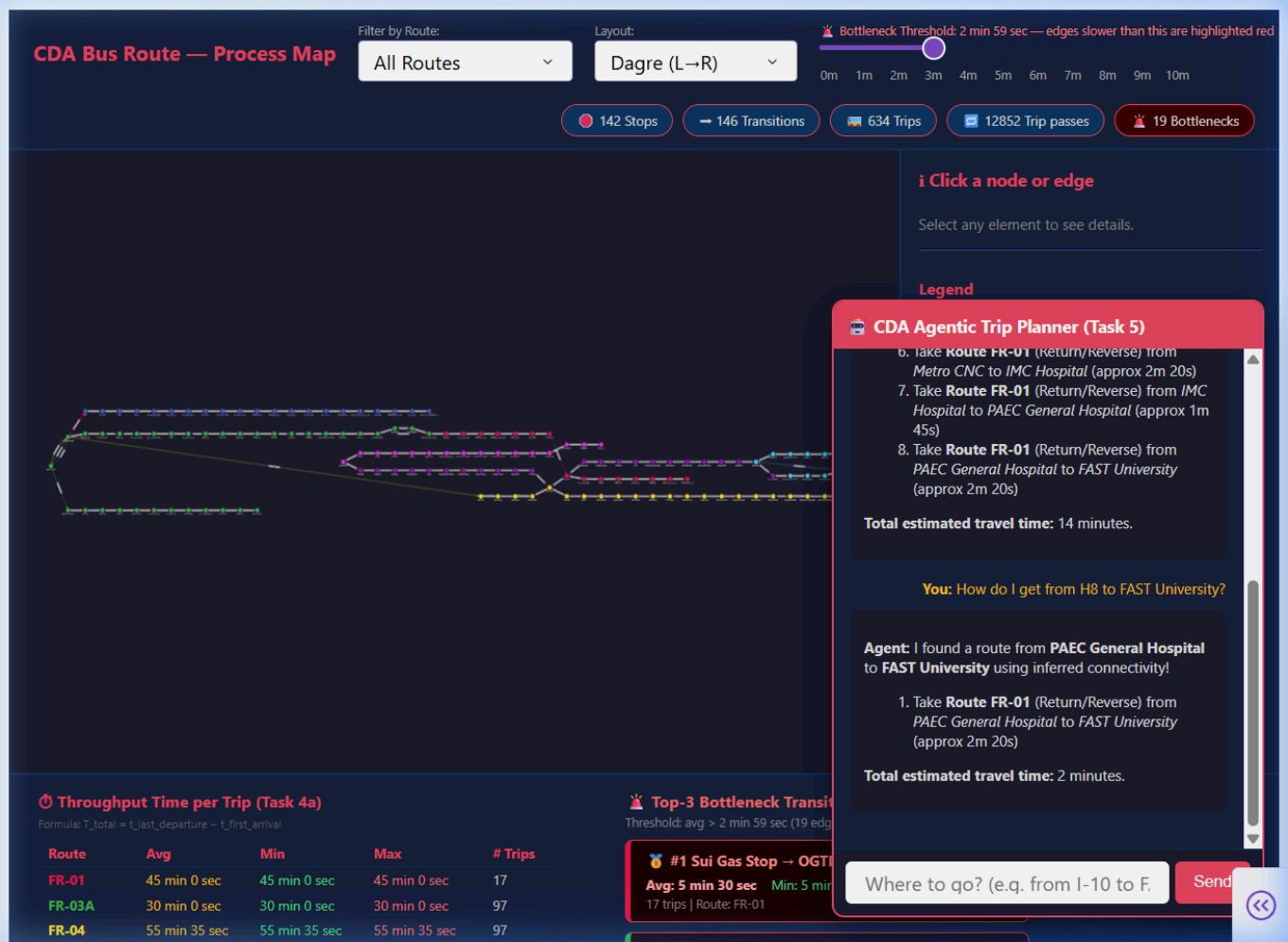


Figure 8: FR-01 reverse trip from PAEC General Hospital (H-8) to FAST University.

Summary Table

Member	Home Area	Nearest Stop	Leg 1	Transfer	Leg 2
Member 1	I-10	PTCL I-10	FR-01 (Reverse) → FAST	—	—
Member 2	Khanna Pul	Khanna Pul	FR-09 (Forward) → Mandi Morh	Mandi Morh	FR-01 (Reverse) → FAST
Member 3	H-8	PAEC General Hospital	FR-01 (Reverse) → FAST	—	—
Member 4	H-8	NORI Hospital	FR-01 (Reverse) → FAST	—	—

8. Limitations and Design Decisions Worth Noting

A few things we want to be upfront about:

Timestamps are synthetic. The CDA PDFs only provide relative schedules, not real calendar dates. Anchoring everything to 2024-01-01 was necessary to get valid `datetime` objects, but it means any day-of-week or multi-day analysis would be meaningless on this dataset.

Min = Max = Avg is expected, not broken. Because every trip on a given route uses the same template schedule, all trips have identical durations. We intentionally explain this in the GUI rather than hiding it.

Return trips are inferred, not sourced. The AI agent's bidirectional edges are synthetic — generated from the forward-only PDFs we were given. We believe this is a reasonable enhancement given the real-world context, but it is worth noting that these return schedules are not validated against actual CDA return timetables.

FR-08C and FRG-1 appear in the colour map but were absent from the PDFs we received. Their colour assignments remain in the code in case the data is extended later.

9. Conclusion

We covered the full process mining pipeline on real CDA data: raw PDF extraction, XES log construction with pm4py, interactive process map visualisation, bottleneck and throughput analytics, and an AI trip planner that reasons over actual route data rather than guessing. The system correctly plans routes for all four group members and handles area aliases, multi-hop transfers, and bidirectional inference.

The biggest technical insight from the analysis was how centrally FR-01 functions in the network — it serves FAST University and acts as the connecting backbone for routes coming from multiple directions. The performance analytics confirmed that the H-8 to I-8 corridor on FR-01 and FR-07 consistently carries the slowest transitions, which lines up with what you'd expect from Islamabad's congestion patterns in that area.